AGENTIC ATSC LAB · STUDY GUIDE

Build traffic-signal agents in SUMO, one lab at a time

Six short Python labs: a first agent, talking agents, a learning agent, eco-driving connected cars, priority for buses and ambulances, and a language-model agent behind a safety shield.

Go 2 · Build in SUMO



INSIDE

- Why every controller must go through the same small door
- An agent's decision worked by hand, reason and all
- One Q-learning update, step by step
- Why a promise of 8 seconds helps cars but costs the signal

Beginner-friendly companion to smrahmanbd.github.io/agentic-atsc-lab

The map: AATSC Go 2

This guide is for beginners who have read Go 1 or know what a traffic-signal agent is. You need a little Python and school arithmetic. You will run six labs in the SUMO traffic simulator and learn to read the code behind each one: what the agent sees, how it decides, and what the results really show, including where it does not win. There are seven ideas: the set-up, then one per lab. Each builds on the one before.

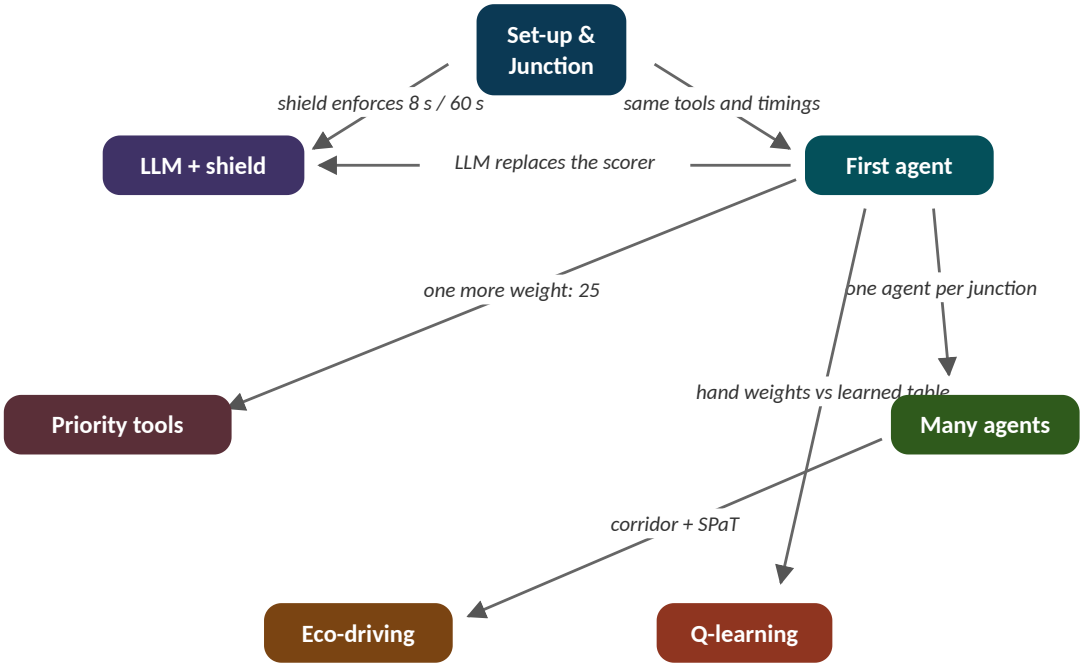


Figure 0. The whole guide on one page. Each box is one concept section; the arrows say how one idea feeds the next. Come back to this page whenever you feel lost.

1. Set-up and the Junction class
2. Lab 1: your first agent
3. Lab 2: many agents on a corridor
4. Lab 3: a learning agent (Q-learning)
5. Lab 4: eco-driving with SPaT and GLOSA
6. Lab 5: tools for priority
7. Lab 6: an LLM agent behind a safety shield

HOW TO USE THIS GUIDE

Read one concept per sitting (20 to 30 minutes), ideally with the lab running beside you. Do every **Simple calculation** with a pencil before reading the answer. Say the **Picture this** scene out loud while you imagine walking down Workshop Lane: the sillier it feels, the better it sticks. The **Recap so far** box at the end of each section grows as you go; reading it is your free revision. When you finish, take the self-quiz, then follow the revision plan on the last page.

What you will be able to do

- Install the lab, run a SUMO simulation from Python and explain why all controllers share one Junction class.
- Compute an agent's keep-or-switch decision from an observation, goal weights and hysteresis.
- Explain how platoon messages between neighbouring agents work and why they cut stops but not delay in Lab 2.
- Carry out a Q-learning update by hand and say why the learner did not yet beat the rules.
- Work out a GLOSA speed advice and explain why an adaptive signal must commit to give one.
- Show how a priority tool and one weight change decisions, and read the bus, ambulance and general delay trade-off.
- Trace an LLM answer through the safety shield and say what the shield can and cannot catch.

Before we start

HOOK

The cleverest-sounding controller left a car waiting 301 s. In Lab 1 max-pressure, a well-known efficient rule, ignored how long the side street had waited. One car waited 301 s before SUMO removed it. The agent, with a small weight on waiting time and a 90 s cap, never let anyone wait more than 20 s.

HOOK

Several results here are honest losses. Talking agents did not beat actuated control on delay; the learning agent did not beat the rules; a promise made for connected cars cost the signal delay. Knowing where a method does not win is part of learning it.

HOOK

The language model is a mock, on purpose. Lab 6 runs offline with a pretend model that gives bad answers 20% of the time. That makes the lab repeatable and lets you watch the safety shield do its job. A real model can be plugged in with one flag.

Words you will meet

You do not need to memorise these now. Each one is explained again when it first matters.

Word	In plain words
SUMO	A free traffic simulator that moves every vehicle second by second.
TraCI	SUMO's remote control: Python reads the traffic and sets the lights through it.
Stage	A set of movements green together. Here: EW (main road) and NS (side street).
Agent	A controller that perceives, reasons and acts toward goals, and explains itself.
Observation	What the agent sees at one moment: queues, waits, arrivals, messages.
Delay	Time a trip lost compared with driving freely, averaged per vehicle.
Platoon	A bunch of cars travelling together, usually released by the previous green.
CAV	Connected (and automated) vehicle: a car that can receive signal timing and advice.
LLM	Large language model: software that reads and writes text, such as a chatbot.

CONCEPT 1 OF 7 · AATSC GO 2

Set-up and the Junction class

HOOK

Six labs compare fixed plans, actuated signals, rule agents, a learner and a language model. How can such different controllers be compared fairly? By forcing every one of them through the same small door.



IN PLAIN WORDS

Think of a cooking contest where every chef gets the same oven, the same knives and the same timer. Whatever dish wins, you know it won on the recipe, not because one chef had a better stove. *Where the picture breaks:* chefs can still bring their own ingredients; here every controller sees the road only through the same tools, so even the 'ingredients' are shared.

How to run. `git clone https://github.com/smrahmanbd/agentic-atsc-lab.git`, then in `sumo`: `python -m pip install -r requirements.txt` (SUMO comes from pip). `python -m pytest tests -q` checks the code; `python go2/01_first_agent.py --gui` shows Lab 1 in sumo-gui.

SUMO simulates every vehicle; **TraCI** is the remote control Python uses to read it and set the lights. `runner.py` steps SUMO once a second, calls the controller, and turns trip records into delay, stops, CO_2 and longest wait.

The key code idea. Each junction is one `Junction` with two stages, EW (main road) and NS (side street). Controllers change lights only through `Junction.request(stage)`: yellow 3 s, all-red 1 s, then green. It also offers the **perception tools** `queue()`, `max_wait()`, `approaching()`, `downstream_occupancy()`, `pressure()`, `priority_requests()`, `cav_estimate()`. Minimum green 8 s and maximum 60 s live in `controllers.py`. Shared timings mean results differ only by decisions.

$$\text{lost time per stage change} = \text{yellow } 3\text{ s} + \text{all-red } 1\text{ s} = 4\text{ s}$$

the same for every controller

$$\text{shortest possible cycle} = 2 \times (\text{min green } 8\text{ s} + 4\text{ s}) = 24\text{ s}$$

two stages, each at minimum green

SIMPLE CALCULATION · What do stage changes cost?

Lab 1's agent made 234 stage changes in its run.

1. Lost time per change = 3 s yellow + 1 s all-red = **4 s**.
2. Total lost = 234 × 4 = **936 s**.
3. As a share of one hour: 936 ÷ 3600 = **0.26**, about a quarter.
4. Shortest cycle: 2 × (8 + 4) = **24 s**.
5. Longest cycle (both at max green): 2 × (60 + 4) = **128 s**.

About a quarter of the hour went on changeovers, so switch only when it clearly pays.

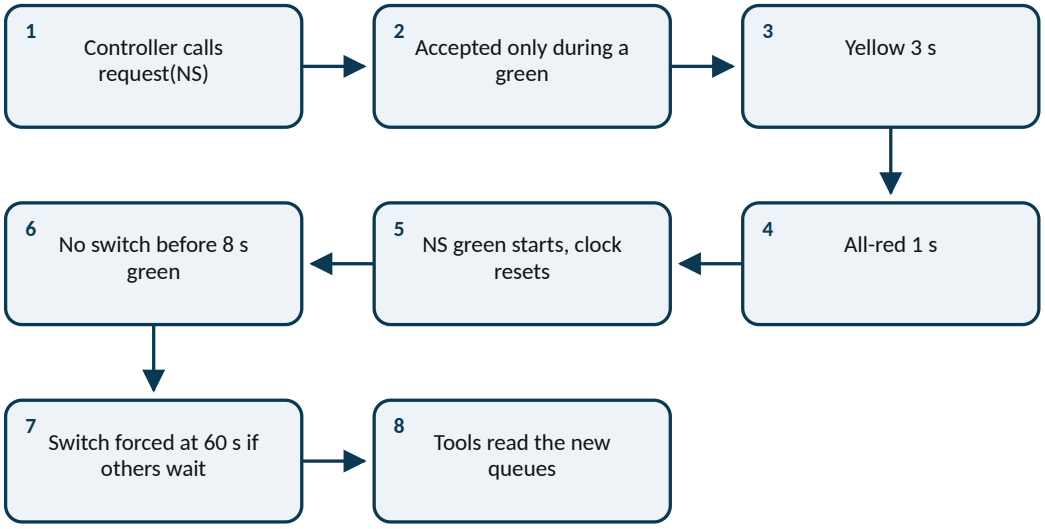


Figure 1. How every controller changes the lights. What to notice: the controller only asks; the Junction class runs the same safe sequence for everyone. Read it left to right, then down.

TRAP

The safety timings are **not** part of any controller's cleverness: nobody can win by skipping yellow.

PICTURE THIS · STOP 1 OF THE ROUTE

A shop sells one toolbox to every robot: inside are a queue ruler, a waiting stopwatch, an arrivals radar, an exit gauge, a VIP detector and blue-car goggles. The shop sign says "pip install".
What it encodes: Sign = pip install; the identical toolbox = the shared Junction tools and safety timings.

TRY IT NOW (about 90 seconds)

Fixed time made 122 stage changes in Lab 1 and actuated control made 254. How much time did each lose to yellow and all-red? *Answer in the key at the back.*

RECAP SO FAR

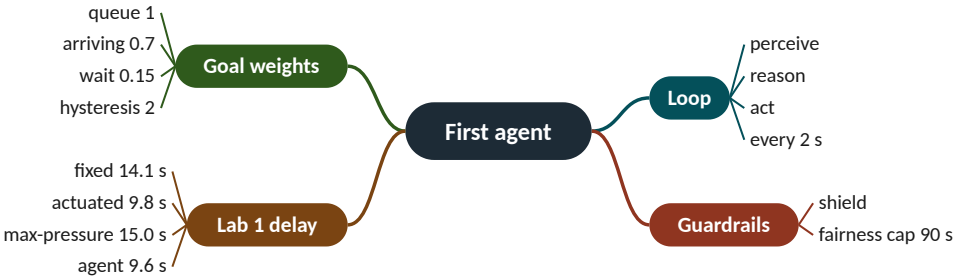
1. **Set-up: one Junction class for all: same tools; yellow 3, all-red 1, green 8-60 s.**

CONCEPT 2 OF 7 · AATSC GO 2

Lab 1: your first agent

HOOK

Your first agent is a few lines of arithmetic: add up what each stage needs, compare, and say why. Can something that simple beat a fixed plan and match a well-tuned actuated signal?



IN PLAIN WORDS

Think of a parent with two children asking for help. They weigh who has been waiting longest, who has the most to do and who is about to arrive, and only stop helping one child when the other clearly needs it more. *Where the picture breaks:* a parent cannot put exact numbers on need; the agent does, with fixed weights, and writes the sum down as its reason.

How to run. `python go2/01_first_agent.py` (add `--gui` to watch). Fixed time, actuated, max-pressure and the agent run on the same one-hour arrivals with a peak.

The key code idea. Every 2 s: `perceive → reason → act`. `reason` scores each stage: 1 × queuing + 0.7 × arriving within 50 m + 0.15 × longest wait (s), plus extras used in later labs. It switches only if the other stage leads by more than the **hysteresis** of 2, and writes its reason. `act` passes the choice through the **safety shield**.

What we got (seed 1): mean delay fixed 14.1 s, actuated 9.8 s, max-pressure 15.0 s, agent 9.6 s. Longest wait: 36, 26, 301 and 20 s. Max-pressure starved the side street (SUMO removed the car after 301 s): it has no notion of fairness. The agent's wait weight and 90 s cap prevent that.

$$\text{score}(\text{stage}) = 1 \times \text{queue} + 0.7 \times \text{arriving} + 0.15 \times \text{longest wait}$$

goals turned into weights (Goals in controllers.py)

$$\text{switch if score}(\text{other}) > \text{score}(\text{current}) + 2$$

hysteresis 2: demand a clear advantage, because each change costs 4 s

SIMPLE CALCULATION · One decision by the agent

EW green 14 s. EW: 3 queuing, 2 arriving, wait 4 s. NS: 7 queuing, 1 arriving, wait 24 s.

1. EW score = 1 × 3 + 0.7 × 2 + 0.15 × 4 = 3 + 1.4 + 0.6 = **5.0**.
2. NS score = 1 × 7 + 0.7 × 1 + 0.15 × 24 = 7 + 0.7 + 3.6 = **11.3**.
3. Bar to beat = current + hysteresis = 5.0 + 2 = **7.0**.
4. 11.3 > 7.0, so the agent proposes **switch**.
5. Shield: green has run 14 s ≥ 8 s minimum, so the switch is allowed.

Log: "switch to NS: score 11.3 vs 5.0 (queue 7, longest wait 24 s)".

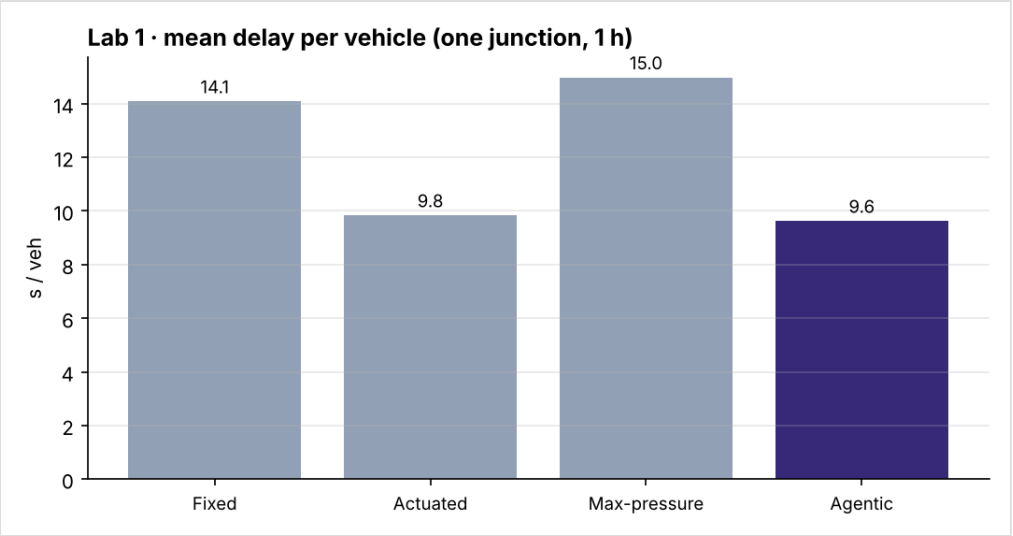


Figure 2. Lab 1: mean delay per vehicle (one junction, one hour). What to notice: the agent (9.6 s) edges actuated control (9.8 s) and cuts a third off fixed time (14.1 s). Max-pressure is worst here because it ignores how long the side street has waited.

TRAP

Lowest mean delay is **not** the whole story: max-pressure's 15.0 s hid a 301 s wait.

PICTURE THIS · STOP 2 OF THE ROUTE

The robot weighs two baskets on a scale every 2 seconds: cars waiting, cars arriving and how long the oldest has waited. A sticky note on the scale reads "9.6".

What it encodes: Scale = scoring each stage with goal weights every 2 s; sticky note = 9.6 s delay.

TRY IT NOW (about 90 seconds)

EW green. EW: 5 queuing, 3 arriving, longest wait 2 s. NS: 6 queuing, 0 arriving, longest wait 30 s. Does the agent switch? What if hysteresis were 6? *Answer in the key at the back.*

RECAP SO FAR

1. **Set-up:** one Junction class for all: same tools; yellow 3, all-red 1, green 8-60 s.
2. **Lab 1: score = 1·queue + 0.7·arriving + 0.15·wait; switch if ahead by 2; 9.6 s.**

CONCEPT 3 OF 7 · AATSC GO 2

Lab 2: many agents on a corridor

HOOK

On a row of four junctions, each agent can see cars leaving its own stop line long before the next junction can. What if it simply told its neighbour?



IN PLAIN WORDS

Think of a relay of neighbours along a street. The first sees a group of children leave school and texts the next house: "six kids, there in twenty seconds". The next one opens the gate just in time instead of guessing. *Where the picture breaks:* the texts here go only downstream, along the main road, and the receiver adds them to a score rather than simply opening the gate.

How to run. `python go2/02_multi_agent.py --seeds 3` (about 10 minutes): fixed green wave, actuated, max-pressure, silent agents and talking agents on four junctions 300 m apart.

The key code idea. In `communicate()` each agent looks at cars heading to its downstream neighbour, estimates arrival as `now + distance ÷ 12 m/s`, and sends a "platoon" `Message` with size and ETA. The receiver adds **0.8 × size** to its EW score when the platoon is due within 12 s.

What we got (mean of 3 seeds): delay 26.5, 17.5, 24.3, 18.4 and 18.2 s; stops 1.11, 0.89, 0.80, 0.95 and 0.88. Talking cut stops by about 7% and CO_2 slightly (232 → 229 g), but actuation still had the lowest delay on this short corridor. Coordination should pay more on longer, denser networks (Go 3).

$$\text{ETA} = \text{now} + \text{remaining distance} \div 12\ \text{m/s}$$

the sender's estimate of arrival at the neighbour

$$\text{EW score} \mathrel{+}= 0.8 \times \text{platoon size, if ETA} \leq \text{now} + 12\ \text{s}$$

w_platoon = 0.8

SIMPLE CALCULATION · A platoon message, start to finish

J1 sees 6 cars 240 m from J2's stop line; travel speed 12 m/s.

1. Travel time = 240 ÷ 12 = **20 s**, so ETA = now + 20 s.
2. 20 s > 12 s window, so J2 stores the message but adds nothing yet.
3. 8 s later the platoon is 12 s away: inside the window.
4. Bonus to J2's EW score = 0.8 × 6 = **4.8** points.
5. Lab 2 stops: (0.95 − 0.88) ÷ 0.95 = **0.074**, about 7% fewer.

A 4.8-point boost holds EW green for the platoon unless NS clearly leads.

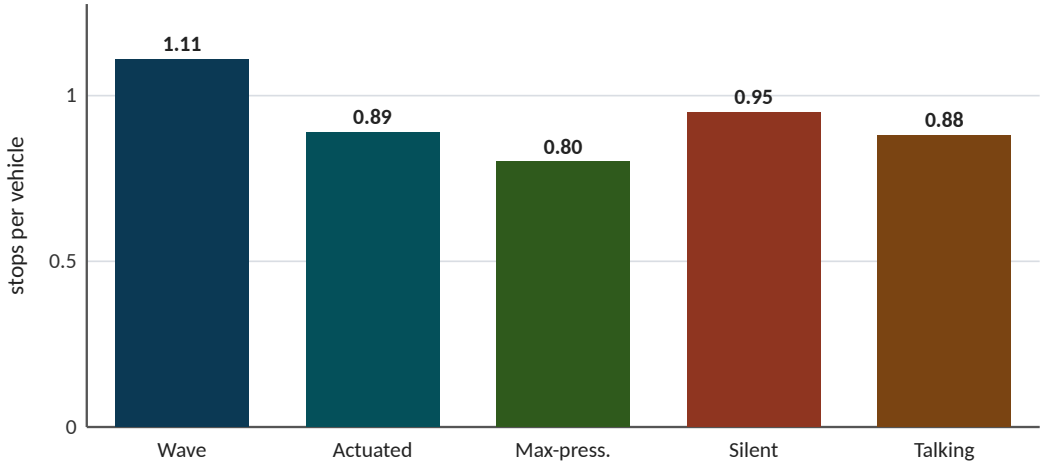


Figure 3. Lab 2: stops per vehicle on the corridor (mean of 3 seeds). What to notice: talking agents stop less often than silent ones (0.88 vs 0.95). Max-pressure has the fewest stops but long waits and high delay. Wave = fixed green wave.

TRAP

Talking agents did **not** win on everything: actuated still had the lowest delay (17.5 vs 18.2 s).

PICTURE THIS · STOP 3 OF THE ROUTE

Four houses in a row pass paper aeroplanes downstream with a number and a time written on each wing.

What it encodes: Houses = four junctions; aeroplanes = platoon messages (size, ETA) sent downstream.

TRY IT NOW (about 90 seconds)

A platoon of 4 cars is 180 m from the next stop line. When is it due, and what bonus does it add once inside the window? *Answer in the key at the back.*

RECAP SO FAR

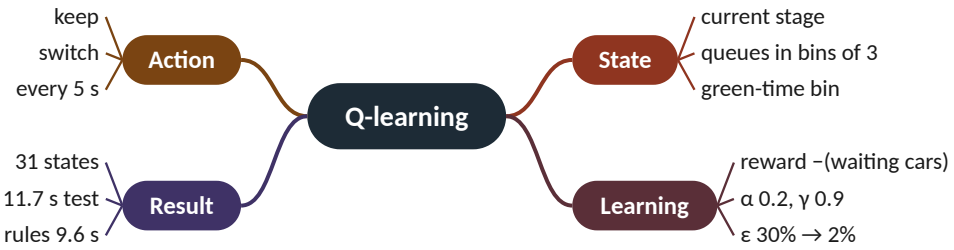
1. **Set-up:** one Junction class for all: same tools; yellow 3, all-red 1, green 8-60 s.
2. **Lab 1:** score = 1·queue + 0.7·arriving + 0.15·wait; switch if ahead by 2; 9.6 s.
3. **Lab 2: platoon size + ETA sent downstream; stops 0.95 → 0.88; actuated lowest delay.**

CONCEPT 4 OF 7 · AATSC GO 2

Lab 3: a learning agent (Q-learning)

HOOK

Instead of writing the weights by hand, could the signal learn when to switch just by trying things and noticing when the queue shrinks?



IN PLAIN WORDS

Think of training a puppy. It tries sitting or jumping; a treat follows good choices. Over many sessions it learns which choice works in which situation. Early on it tries random things on purpose, to discover what earns treats. *Where the picture breaks:* the puppy's treat comes at once; the agent's 'treat' is a number every 5 s, and it must guess how today's choice affects future treats too.

How to run. `python go2/03_learning_agent.py --episodes 30`: 30 training episodes of 30 minutes with fresh arrivals, then a greedy test hour against actuated and the rule agent.

The key code idea. State = current stage, EW and NS queues in bins of 3 cars, and a green-time bin. Every 5 s the agent picks **keep** or **switch**, gets **reward** −(cars waiting), and updates `Q ← Q + α (r + γ · max Q(next) − Q)`, α 0.2, γ 0.9. With probability ε (30% → 2%) it explores at random.

What we got. 31 states visited; 11.7 s on the test hour vs 9.8 s actuated and 9.6 s rules. The training curve is noisy (11.0 to 12.8 s) with no clear downward trend. Tabular learning needs more training, a richer state and reward shaping to beat good rules.

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a) \right]$$

α = 0.2 learning rate, γ = 0.9 discount

$$r = -(\text{queue EW} + \text{queue NS})$$

fewer waiting cars is better

SIMPLE CALCULATION · One Q-learning update

Q(s, keep) = −10. Next, 6 cars wait; best Q in the new state is −12.

1. Reward r = −6.
2. Target = r + γ × best next = −6 + 0.9 × (−12) = −6 − 10.8 = **−16.8**.
3. Error = target − old = −16.8 − (−10) = **−6.8**.
4. Step = α × error = 0.2 × (−6.8) = **−1.36**.
5. New Q(s, keep) = −10 − 1.36 = **−11.36**.

Keep now looks a little worse; only 20% of each surprise is absorbed.

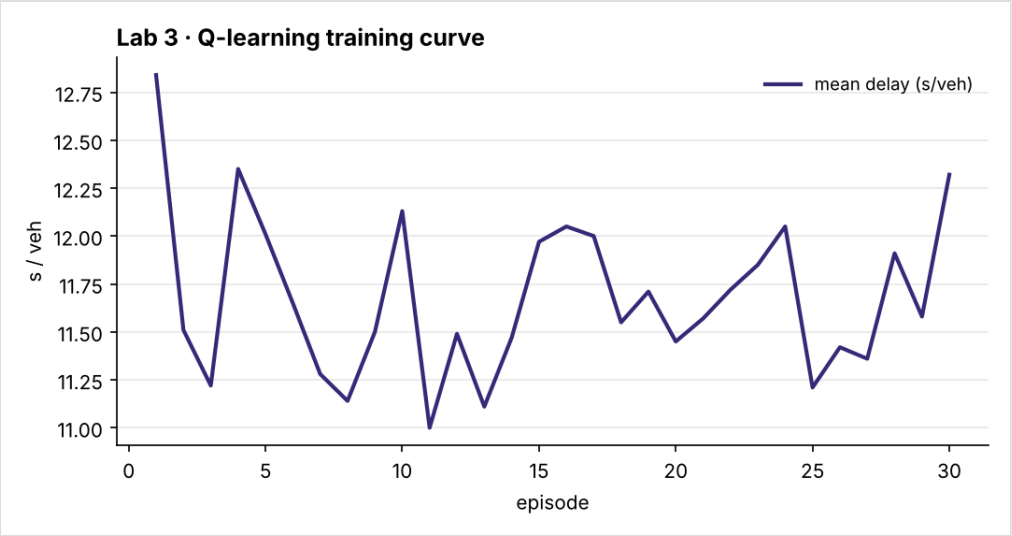


Figure 4. Lab 3: Q-learning training curve (mean delay per episode). What to notice: the curve jumps around 11 to 12.8 s and does not fall steadily. Each episode has new random arrivals and exploration falls from 30% to 2%, so a single episode is a noisy signal of progress.

TRAP

"It learns" does **not** mean "it beats the rules": 11.7 s vs 9.6 s (rules) and 9.8 s (actuated).

PICTURE THIS · STOP 4 OF THE ROUTE

A puppy learns to press "keep" or "switch" and gets a treat each time the queue shrinks; its notebook has 31 pages.

What it encodes: Keep / switch = actions; treat = reward; 31-page notebook = Q-table with 31 states.

TRY IT NOW (about 90 seconds)

Q(s, switch) = 0. After switching, 4 cars are waiting and the best Q in the next state is −5. What is the new Q(s, switch)? *Answer in the key at the back.*

RECAP SO FAR

1. **Set-up:** one Junction class for all: same tools; yellow 3, all-red 1, green 8-60 s.
2. **Lab 1:** score = 1·queue + 0.7·arriving + 0.15·wait; switch if ahead by 2; 9.6 s.
3. **Lab 2:** platoon size + ETA sent downstream; stops 0.95 → 0.88; actuated lowest delay.
4. **Lab 3: Q-learning, reward −waiting, α 0.2, γ 0.9; 31 states; 11.7 s, not yet beating rules.**

CONCEPT 5 OF 7 · AATSC GO 2

Lab 4: eco-driving with SPaT and GLOSA

HOOK

A connected car told "green in 14 seconds" can ease off and glide through instead of braking to a stop. But an adaptive signal does not know its own future. What must it promise?



IN PLAIN WORDS

Think of a ferry that announces "leaving in 5 minutes". You walk calmly instead of sprinting and then waiting. If the ferry never announced its time, you would have to rush and hope. *Where the picture breaks:* a ferry timetable is fixed; an adaptive signal only knows its next change once it decides, so it must commit in advance to give a time at all.

How to run. `python go2/04_eco_driving.py`: a fixed plan and the agent committing 8 s ahead, each with 0, 25, 50 and 100% connected vehicles (CAVs).

The key code idea. The signal publishes **SPaT**: green stage and, if known, seconds to change. In `cav.py` a CAV within 300 m computes the next green (`remaining + yellow + all-red`) and drives at `distance ÷ time`, between 5 m/s and the limit: **GLOSA**. A fixed plan always knows its time; the agent knows it only once it commits (`commit_horizon=8`).

What we got (seed 1): fixed plan, 0 → 100% CAVs: stops 1.12 → 0.50, CO_2 254.4 → 244.3 g. Committed agent: delay 24.0 → 20.0 s, stops 1.00 → 0.85; CO_2 245.8 → 244.0 g, but 248.1 g at 25%. The price: the same agent without commitment had 18.4 s on this corridor and seed (Lab 2 run).

$$\text{time to next green} = \text{remaining} + \text{yellow } 3\text{ s} + \text{all-red } 1\text{ s}$$

when the other stage is green and its end is known

$$\text{advised speed} = \max(5\text{ m/s}, \min(\text{limit}, \text{distance} \div \text{time to next green}))$$

only if driving at the limit would arrive on red

SIMPLE CALCULATION · A GLOSA speed advice

CAV 150 m away, limit 13.89 m/s; NS green, committed to change in 10 s.

1. At the limit it would arrive in 150 ÷ 13.89 = **10.8 s**.
2. EW turns green in 10 + 3 + 1 = **14 s**.
3. 10.8 s < 14 s, so at full speed it would reach a red and stop.
4. Advised speed = 150 ÷ 14 = **10.7 m/s** (about 38.6 km/h).
5. Check: between 5 and 13.89 m/s, so the advice stands.

It eases to about 39 km/h and glides in on green. With no committed time, no advice.

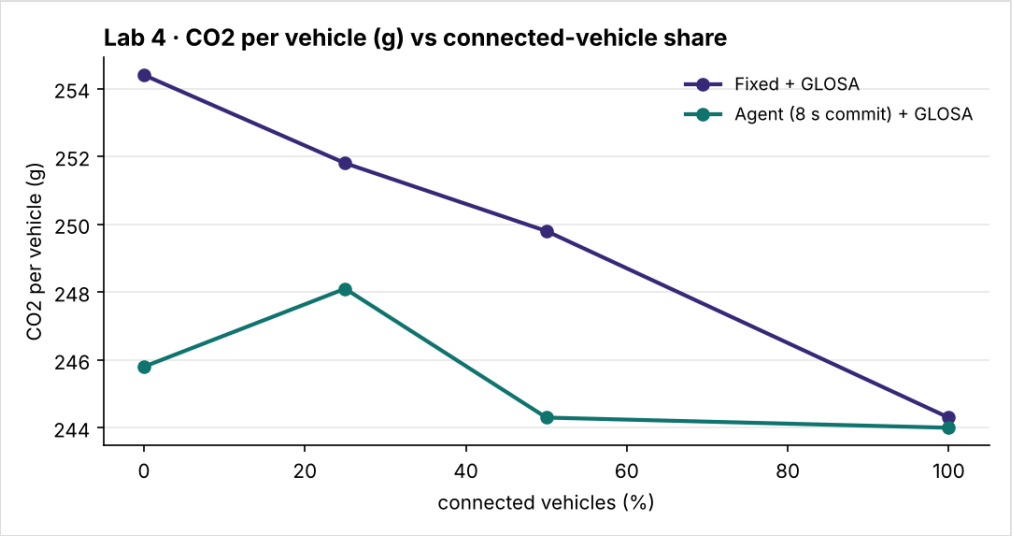


Figure 5. Lab 4: CO2 per vehicle against the share of connected vehicles. What to notice: with the fixed plan, CO2 falls steadily as CAVs increase. The committed agent starts lower but is not monotonic (it rises at 25%). At 100% the two meet near 244 g. One seed only.

TRAP

SPaT is **not** free for an adaptive signal: to announce a time it must commit, and then cannot react.

PICTURE THIS · STOP 5 OF THE ROUTE

The traffic light holds up a promise card "change in 8 s" and a parade of blue swans slows to glide through.

What it encodes: Promise card = SPaT with an 8 s commitment; gliding swans = CAVs following GLOSA.

TRY IT NOW (about 90 seconds)

A CAV is 250 m away (limit 13.89 m/s). The other stage is green with 20 s left. What speed is advised? *Answer in the key at the back.*

RECAP SO FAR

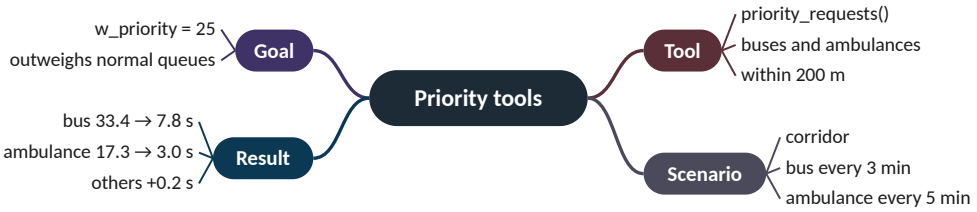
1. **Set-up:** one Junction class for all: same tools; yellow 3, all-red 1, green 8-60 s.
2. **Lab 1:** score = 1·queue + 0.7·arriving + 0.15·wait; switch if ahead by 2; 9.6 s.
3. **Lab 2:** platoon size + ETA sent downstream; stops 0.95 → 0.88; actuated lowest delay.
4. **Lab 3:** Q-learning, reward −waiting, α 0.2, γ 0.9; 31 states; 11.7 s, not yet beating rules.
5. **Lab 4: SPaT + GLOSA, v = d ÷ (remaining + 4 s); agent must commit 8 s, which costs delay.**

CONCEPT 6 OF 7 · AATSC GO 2

Lab 5: tools for priority

HOOK

A full bus or an ambulance should not wait behind a queue of cars. One new tool and one large weight change the agent's mind. What does it cost everyone else?



IN PLAIN WORDS

Think of a supermarket that opens a till the moment a customer with a priority card appears. Others barely notice, because it happens rarely, but the card holder walks straight through. *Where the picture breaks:* the shop opens an extra till; the junction has no extra green to give, so priority is taken from the other stage, within the shield's rules.

How to run. `python go2/05_priority.py`: corridor, eastbound bus every 3 min, ambulance every 5 min; actuated vs agent without priority (`Goals(w_priority=0)`) vs agent with its tool.

The key code idea. `Junction.priority_requests(stage)` lists buses and ambulances within 200 m. If any, the agent adds `w_priority = 25` to that stage's score, more than a typical queue, so it switches or holds for the vehicle and names it in its reason. The shield still applies.

What we got: bus delay 33.4, 29.1 and 7.8 s; ambulance 17.3, 17.8 and 3.0 s; everyone 17.8, 18.2 and 18.4 s. Bus delay fell by about three-quarters, at about 0.2 s for everyone else.

$$\text{score}(\text{stage}) \mathrel{+}= 25 \text{ if a bus or ambulance is within } 200 \text{ m}$$

w_priority = 25

$$\text{relative reduction} = (\text{before} - \text{after}) \div \text{before}$$

how much of the delay was removed

SIMPLE CALCULATION · The VIP pass beats the queue

NS green 20 s: 12 queuing, 3 arriving. EW: 2 queuing, wait 10 s. A bus appears on EW.

1. NS = 12 + 0.7 × 3 = **14.1**; EW = 2 + 0.15 × 10 = **3.5**: keep.
2. EW with the bus = 3.5 + 25 = **28.5**.
3. 28.5 > 14.1 + 2 = 16.1, so **switch** (20 s ≥ 8 s, shield allows).
4. Bus delay vs actuated: (33.35 − 7.82) ÷ 33.35 = **0.77**.

One weight turns keep into switch. Vs the agent without priority the cut is (29.11 − 7.82) ÷ 29.11 = 0.73.

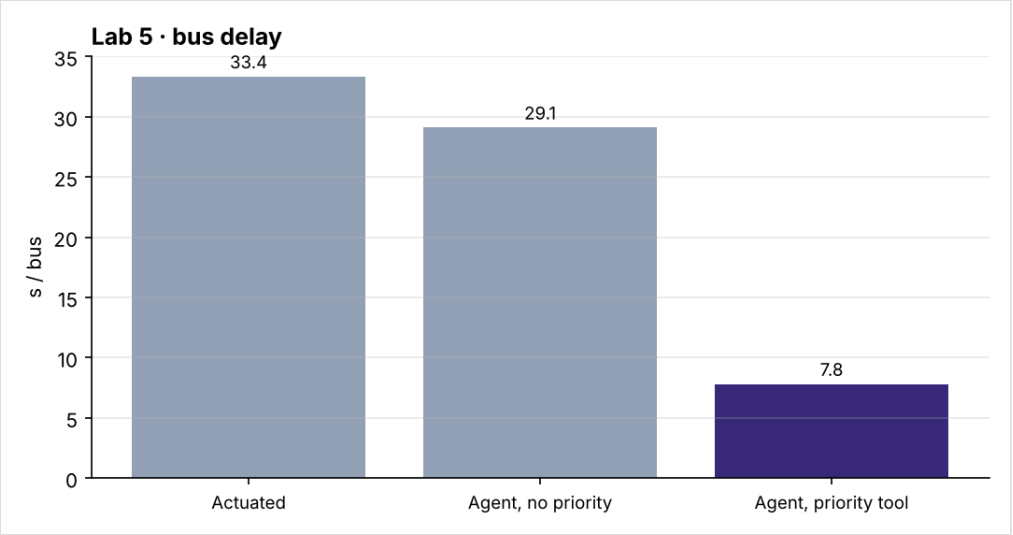


Figure 6. Lab 5: mean bus delay. What to notice: removing the priority tool leaves the agent close to actuated control; switching it on cuts bus delay from about 30 s to 7.8 s. Everyone else paid about 0.2 s.

TRAP

Priority is **not** free: everyone else +0.2 s, and the shield still blocks a switch before 8 s green.

PICTURE THIS · STOP 6 OF THE ROUTE

A yellow bus and an ambulance flash a golden VIP pass worth 25 points; the lights part like a red carpet.

What it encodes: VIP pass = priority_requests() plus w_priority 25; red carpet = bus 33 → 8 s, ambulance 17 → 3 s.

TRY IT NOW (about 90 seconds)

Ambulance delay: 17.28 s actuated, 3.0 s with the priority tool. What fraction was removed? *Answer in the key at the back.*

RECAP SO FAR

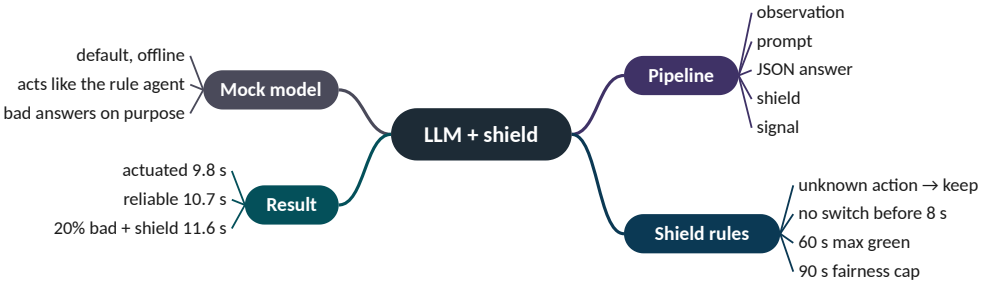
1. **Set-up:** one Junction class for all: same tools; yellow 3, all-red 1, green 8-60 s.
2. **Lab 1:** score = 1·queue + 0.7·arriving + 0.15·wait; switch if ahead by 2; 9.6 s.
3. **Lab 2:** platoon size + ETA sent downstream; stops 0.95 → 0.88; actuated lowest delay.
4. **Lab 3:** Q-learning, reward −waiting, α 0.2, γ 0.9; 31 states; 11.7 s, not yet beating rules.
5. **Lab 4:** SPaT + GLOSA, v = d ÷ (remaining + 4 s); agent must commit 8 s, which costs delay.
6. **Lab 5: priority within 200 m adds 25; bus 33.4 → 7.8 s; everyone else +0.2 s.**

CONCEPT 7 OF 7 · AATSC GO 2

Lab 6: an LLM agent behind a safety shield

HOOK

A language model can read the junction's situation and answer in plain words. It can also answer nonsense with total confidence. How do you let it near a traffic light?



IN PLAIN WORDS

Think of a talkative parrot giving orders on a ship, with a calm officer who checks each order against the rulebook. Sensible orders pass; dangerous ones are replaced with "hold course". *Where the picture breaks:* the officer cannot tell a merely unhelpful order from a wise one; the shield blocks only what breaks a rule, so silly but legal answers still cost delay.

How to run. `python go2/06_llm_agent.py` uses an offline **mock** model (repeatable). With `pip install anthropic` and an API key, add `--backend anthropic --minutes 10` for a real model.

The key code idea. Every 5 s: observation → **prompt** (plus goals) → JSON `{"action": "keep" | "switch", "reason"}` → **shield** → `Junction.request`. The mock acts like the rule agent but, with probability `hallucination`, answers badly: switching too early, never switching, inventing a stage ("green_NE") or holding all-red. The shield turns unknown actions into keep, blocks switches before 8 s, and forces a switch at 60 s green or a 90 s wait.

What we got: actuated 9.8 s; reliable mock 10.7 s; 20% bad answers + shield 11.6 s. 165 of 735 answers were bad; no invented stage or all-red reached the signal. Even the reliable run had 97 blocks: the rules do not check minimum green, the shield does. Go 3 R2 removes the shield.

$$\text{observation} \rightarrow \text{prompt} \rightarrow \text{JSON answer} \rightarrow \text{shield} \rightarrow \text{Junction.request}$$

the model never touches the signal directly

$$\text{unknown action} \rightarrow \text{keep; switch with green} < 8\ \text{s} \rightarrow \text{keep}$$

plus forced switch at 60 s green or 90 s wait

SIMPLE CALCULATION · What did the shield cost and catch?

Actuated 9.84 s; reliable mock 10.73 s; 20% bad + shield 11.58 s; 165 bad of 735 calls.

1. Share of bad answers = 165 ÷ 735 = **0.224**, about 22% (set to 20%, drawn at random).
2. Reliable mock vs actuated: 10.73 − 9.84 = **0.89 s** more (+9%).
3. Bad answers + shield vs actuated: 11.58 − 9.84 = **1.74 s** more (+18%).
4. Bad answers cost a further 11.58 − 10.73 = **0.85 s**.
5. Unsafe orders reaching the signal: **0**.

One bad answer in five: still safe, and under a second of extra delay.

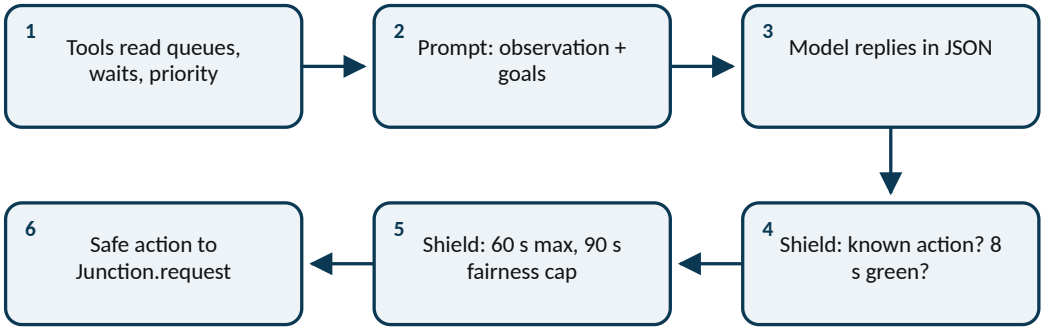


Figure 7. From model answer to traffic light. What to notice: the shield sits between the model and the signal, outside the model, and checks every answer. Read it left to right, then down.

TRAP

The shield does **not** make a bad model good: legal but poor answers pass and cost delay.

PICTURE THIS · STOP 7 OF THE ROUTE

A talkative parrot shouts orders, some of them nonsense ("green to the north-east!"); a guard reads every order against the rulebook before passing it on.

What it encodes: Parrot = the LLM; nonsense = bad answers; guard = the shield checking every answer.

TRY IT NOW (about 90 seconds)

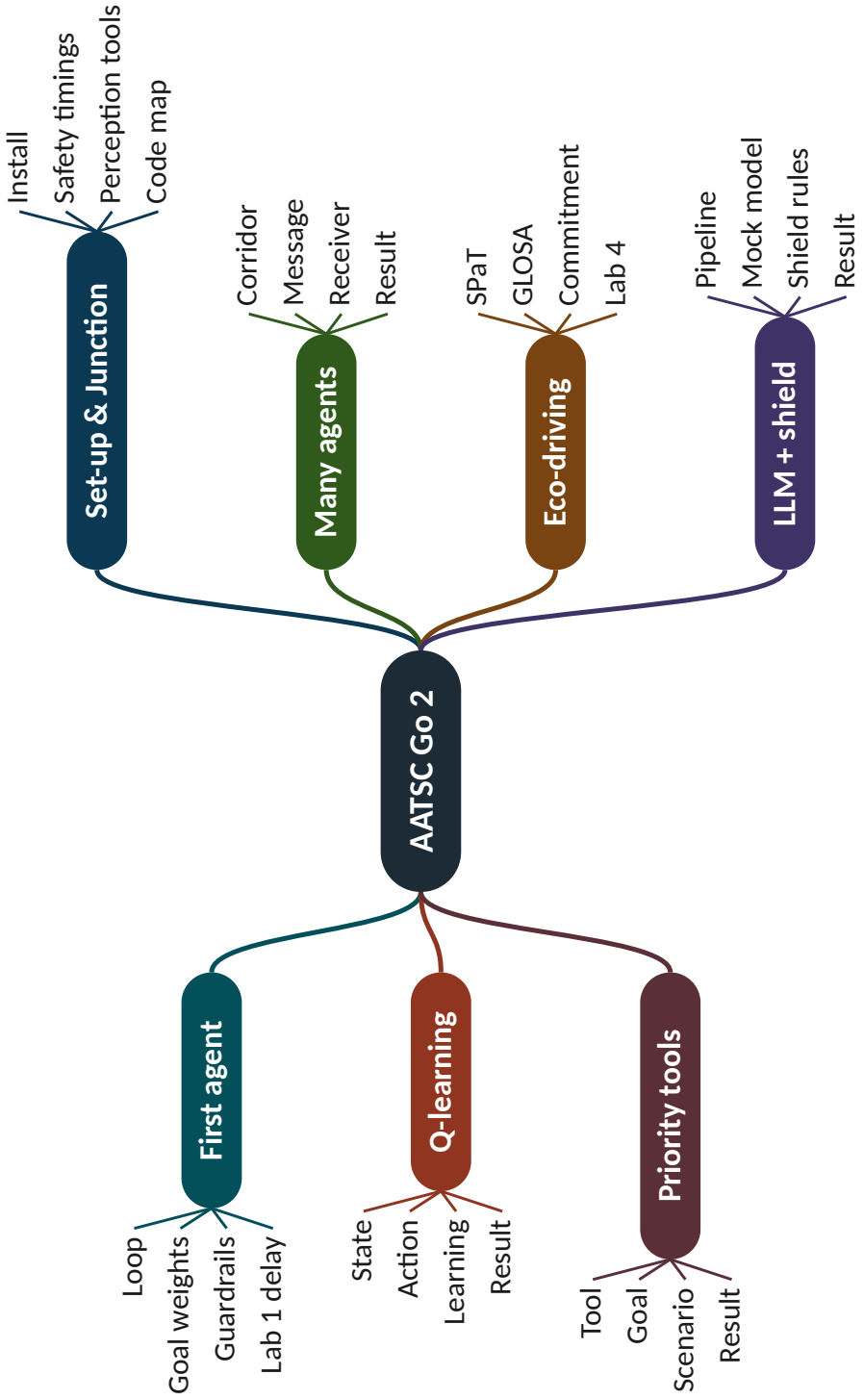
EW has been green for 6 s. The model answers {"action": "switch"}. Then, at 30 s of green, it answers {"action": "green_NE"}. What does the shield do each time? *Answer in the key at the back.*

RECAP SO FAR

1. **Set-up:** one Junction class for all: same tools; yellow 3, all-red 1, green 8-60 s.
2. **Lab 1:** score = 1·queue + 0.7·arriving + 0.15·wait; switch if ahead by 2; 9.6 s.
3. **Lab 2:** platoon size + ETA sent downstream; stops 0.95 → 0.88; actuated lowest delay.
4. **Lab 3:** Q-learning, reward −waiting, α 0.2, γ 0.9; 31 states; 11.7 s, not yet beating rules.
5. **Lab 4:** SPaT + GLOSA, v = d ÷ (remaining + 4 s); agent must commit 8 s, which costs delay.
6. **Lab 5:** priority within 200 m adds 25; bus 33.4 → 7.8 s; everyone else +0.2 s.
7. **Lab 6: prompt → JSON → shield → signal; mock LLM; 20% bad answers, 11.6 s, none unsafe.**

The master mind map

Every concept and its main branches on one page. Turn the page sideways. Try redrawing it from memory on blank paper, then compare.



Connections and contrasts

The controllers you build in Go 2

	Actuated	Rule agent	Q-learning	LLM + shield
Decides with	gaps between vehicles	goal weights and hysteresis	a learned table of keep/switch values	a model's JSON answer
How often	every second	every 2 s	every 5 s	every 5 s
Explains itself?	no	yes, a reason per decision	no	yes, the model's reason
Safety	min / max green	shield	min green only	shield
One-junction delay	9.8 s	9.6 s	11.7 s	10.7 s (11.6 s with 20% bad)

Things beginners mix up

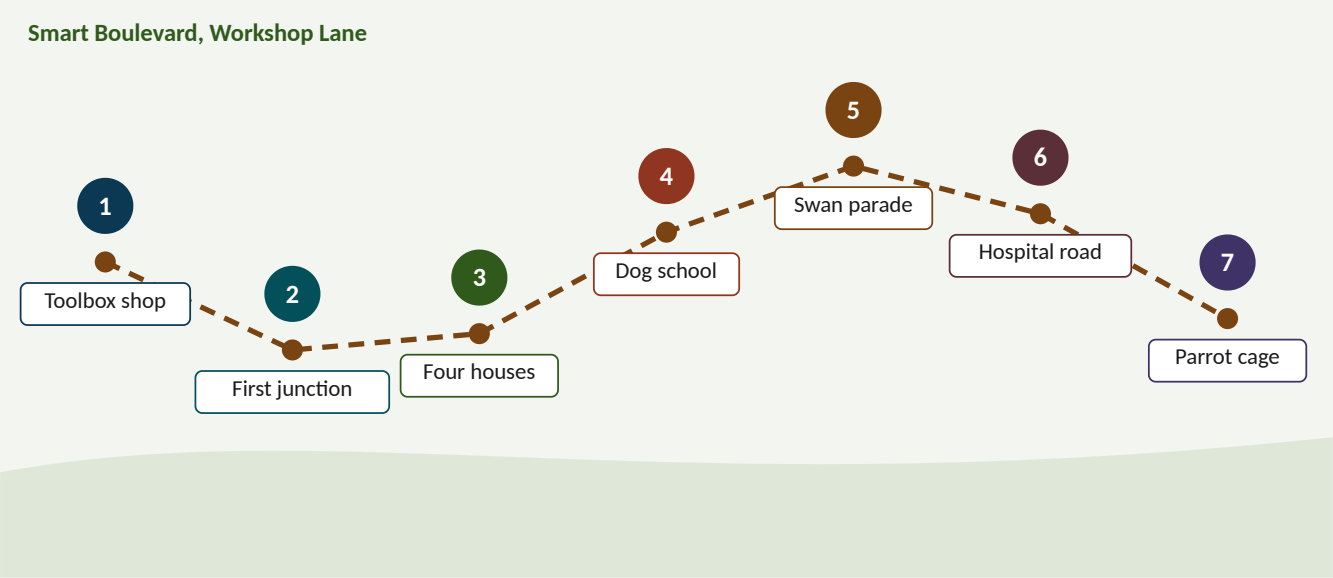
This...	...is not this	The one fact that tells them apart
Hysteresis	**Minimum green**	Hysteresis (2 points) is a judgement margin; minimum green (8 s) is a hard rule the shield enforces.
Green wave	**Platoon messages**	A green wave fixes offsets in advance; messages tell the neighbour live what is coming.
SPaT	**GLOSA**	SPaT is what the signal broadcasts; GLOSA is the speed the car works out from it.
Bad answer	**Unsafe action**	The shield blocks rule-breaking answers; legal but poor answers pass and cost delay.
Mock LLM	**Real LLM**	Lab 6 numbers come from a mock that imitates failures; a real model needs the anthropic backend option.

Symbols and terms

Symbol / term	Meaning
Junction.request(stage)	the only way to change lights: yellow 3 s, all-red 1 s, then green
Perception tool	a Junction method that reads SUMO, e.g. queue(), max_wait(), priority_requests()
Goal weights	w_queue 1, w_arrive 0.7, w_wait 0.15, w_platoon 0.8, w_priority 25
Hysteresis	margin (2 points) the other stage must win by before a switch
Safety shield	checks every proposed action against hard rules before the signal
α, γ, ε	Q-learning rate 0.2, discount 0.9, exploration 30% → 2%
SPaT	Signal Phase and Timing: green stage and seconds to change
GLOSA	Green Light Optimal Speed Advisory: speed to arrive on green
Commit horizon	how far ahead the agent fixes a change so it can announce it (8 s)

The route: Smart Boulevard, Workshop Lane

Walk on from Old Town down Workshop Lane, from a toolbox shop to a parrot cage: one odd, loud scene per concept.



The memory route. Walk it in order; each stop holds one concept.

Stop 1: Toolbox shop (Set-up and the Junction class). A shop sells one toolbox to every robot: inside are a queue ruler, a waiting stopwatch, an arrivals radar, an exit gauge, a VIP detector and blue-car goggles. The shop sign says "pip install". *Sign = pip install; the identical toolbox = the shared Junction tools and safety timings.*

Stop 2: First junction (Lab 1: your first agent). The robot weighs two baskets on a scale every 2 seconds: cars waiting, cars arriving and how long the oldest has waited. A sticky note on the scale reads "9.6". *Scale = scoring each stage with goal weights every 2 s; sticky note = 9.6 s delay.*

Stop 3: Four houses (Lab 2: many agents on a corridor). Four houses in a row pass paper aeroplanes downstream with a number and a time written on each wing. *Houses = four junctions; aeroplanes = platoon messages (size, ETA) sent downstream.*

Stop 4: Dog school (Lab 3: a learning agent (Q-learning)). A puppy learns to press "keep" or "switch" and gets a treat each time the queue shrinks; its notebook has 31 pages. *Keep / switch = actions; treat = reward; 31-page notebook = Q-table with 31 states.*

Stop 5: Swan parade (Lab 4: eco-driving with SPaT and GLOSA). The traffic light holds up a promise card "change in 8 s" and a parade of blue swans slows to glide through. *Promise card = SPaT with an 8 s commitment; gliding swans = CAVs following GLOSA.*

Stop 6: Hospital road (Lab 5: tools for priority). A yellow bus and an ambulance flash a golden VIP pass worth 25 points; the lights part like a red carpet. *VIP pass = priority_requests() plus w_priority 25; red carpet = bus 33 → 8 s, ambulance 17 → 3 s.*

Stop 7: Parrot cage (Lab 6: an LLM agent behind a safety shield). A talkative parrot shouts orders, some of them nonsense ("green to the north-east!"); a guard reads every order against the rulebook before passing it on. *Parrot = the LLM; nonsense = bad answers; guard = the shield checking every answer.*

WALK-THROUGH DRILL

Close the guide. Walk the route in your head, saying each stop out loud and the one number or formula it holds. Then check against the cheat sheet. Repeat until you can do it in under two minutes.

Self-quiz

18 questions: warm-up (recall), core (use the ideas), challenge (combine them). Answer on paper first, then check the key.

Warm-up

Q1. In the lab's Junction class, how long are yellow and all-red?

A. 2 s and 2 s
B. 3 s and 1 s
C. 4 s and 0 s
D. 5 s and 2 s

Q2. What three steps does the Lab 1 agent repeat every 2 s?

A. Measure, store, report
B. Perceive, reason, act
C. Predict, optimise, broadcast
D. Train, test, deploy

Q3. What does an agent tell its downstream neighbour in Lab 2?

A. Its cycle length
B. Its current queue on the side street
C. The size and arrival time of a platoon
D. The CO2 it has saved

Q4. What does GLOSA stand for?

A. Green Light Optimal Speed Advisory
B. Global Load Optimisation for Signal Arrays
C. Green Lane Occupancy Sensing Algorithm
D. Guided Lane Overtaking Safety Assistant

Q5. Which model answers in Lab 6 by default?

A. A real cloud LLM
B. A mock model that runs offline
C. A Q-learning table
D. A human operator

Core

Q6. Why does every controller change the lights only through Junction.request?

A. It is faster than TraCI
B. So all share the same safety timings and comparisons are fair
C. Because SUMO allows only one call per second
D. To save CO2

Q7. A stage has 4 cars queuing, 2 arriving and a longest wait of 10 s. What is its score with the Lab 1 weights?

A. 5.4
B. 6.9
C. 7.5
D. 16.0

Q8. Why did max-pressure produce a 301 s wait in Lab 1?

A. SUMO crashed
B. It switches too often
C. It has no notion of fairness, so the side street was starved
D. Its yellow time was longer

Q9. Which controller had the lowest mean delay on the Lab 2 corridor?

A. Fixed green wave
B. Actuated
C. Agents, talking
D. Max-pressure

Q10. Q = −4, reward −2, best next Q = −5, α = 0.2, γ = 0.9. What is the new Q?

A. −4.5
B. −5.0
C. −6.5
D. −3.5

Q11. Why can't an adaptive agent with no commitment support GLOSA?

A. Its SPaT has no remaining time, so no advice can be computed
B. It does not know which stage is green
C. CAVs ignore adaptive signals
D. GLOSA works only at night

Q12. What does the Lab 5 priority tool add to a stage's score, and how far does it look?

A. +2 within 50 m
B. +8 within 300 m
C. +25 within 200 m
D. +90 within 100 m

Q13. Compared with the same agent without priority, how much did everyone's mean delay change in Lab 5?

A. Fell by 5 s
B. No change at all
C. Rose by about 0.2 s
D. Rose by about 10 s

Q14. The model answers {"action": "green_NE"}. What does the shield do?

A. Turns NE green
B. Holds all-red
C. Replaces it with keep
D. Stops the simulation

Challenge

Q15. In the worked example (NS 11.3 vs EW 5.0, EW green), what would the agent do with hysteresis 6?

A. Keep: the margin is now too big
B. Switch: 11.3 is above 11.0
C. Keep: 11.3 is below 13.0
D. Switch only at 60 s of green

Q16. Lab 3's state is stage (2) × EW bin (6) × NS bin (6) × green bin (4). How many states are possible, and roughly what share was visited?

A. 48, about two-thirds
B. 144, about a fifth
C. 288, about a tenth
D. 1024, about 3%

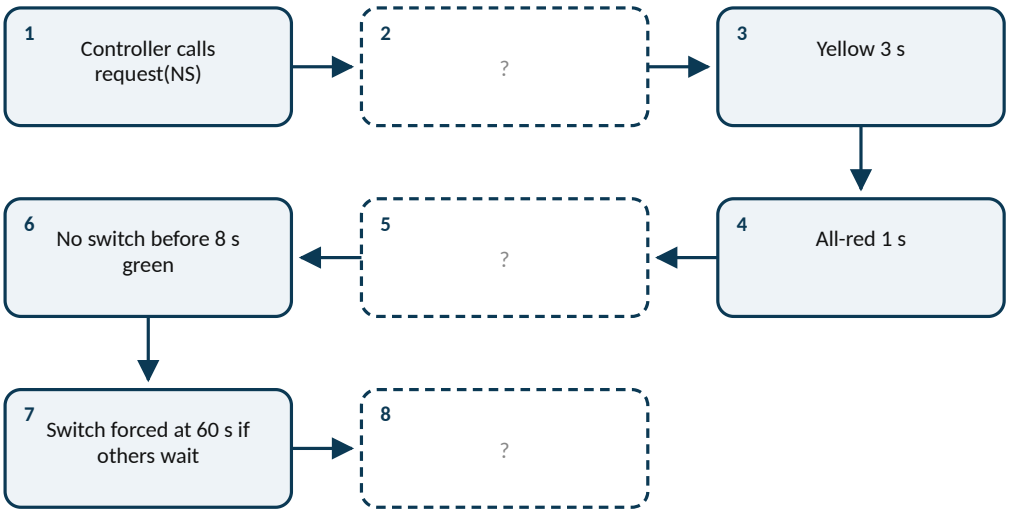
Q17. A CAV is 200 m away (limit 13.89 m/s); the other stage is green with 16 s left. What speed is advised?

A. 13.9 m/s (no advice)
B. 12.5 m/s
C. 10.0 m/s
D. 5.0 m/s

Q18. In Lab 6 the reliable mock (no bad answers) still had 97 answers blocked. Why?

A. The shield is faulty
B. The rule reasoning can propose a switch before 8 s of green; the shield enforces it
C. The model invented stages
D. The fairness cap was set to 10 s

Draw it. Fill the blank boxes of this flow from memory (it is Figure 1).



Where to go if you missed a question

Missed	Do this
Q1, Q6	Re-read concept 1: Set-up and the Junction class
Q2, Q7, Q8, Q15	Re-read concept 2: Lab 1: your first agent
Q3, Q9	Re-read concept 3: Lab 2: many agents on a corridor
Q10, Q16	Re-read concept 4: Lab 3: a learning agent (Q-learning)
Q4, Q11, Q17	Re-read concept 5: Lab 4: eco-driving with SPaT and GLOSA
Q12, Q13	Re-read concept 6: Lab 5: tools for priority
Q5, Q14, Q18	Re-read concept 7: Lab 6: an LLM agent behind a safety shield

Make it stick

BUILD THE SMALLEST WORKING EXAMPLE (15 min)

On paper, take one junction with EW green for 10 s. EW: 4 queuing, 1 arriving, longest wait 0 s. NS: 5 queuing, 0 arriving, longest wait 20 s. Score both stages, decide, and pass the decision through the shield. Then add a bus on NS and redo it.

Self-check: EW = 4 + 0.7 + 0 = 4.7; NS = 5 + 3 = 8.0. 8.0 > 6.7, so switch; 10 s ≥ 8 s, the shield allows it. With the bus, NS = 33.0: still switch, by a much wider margin.

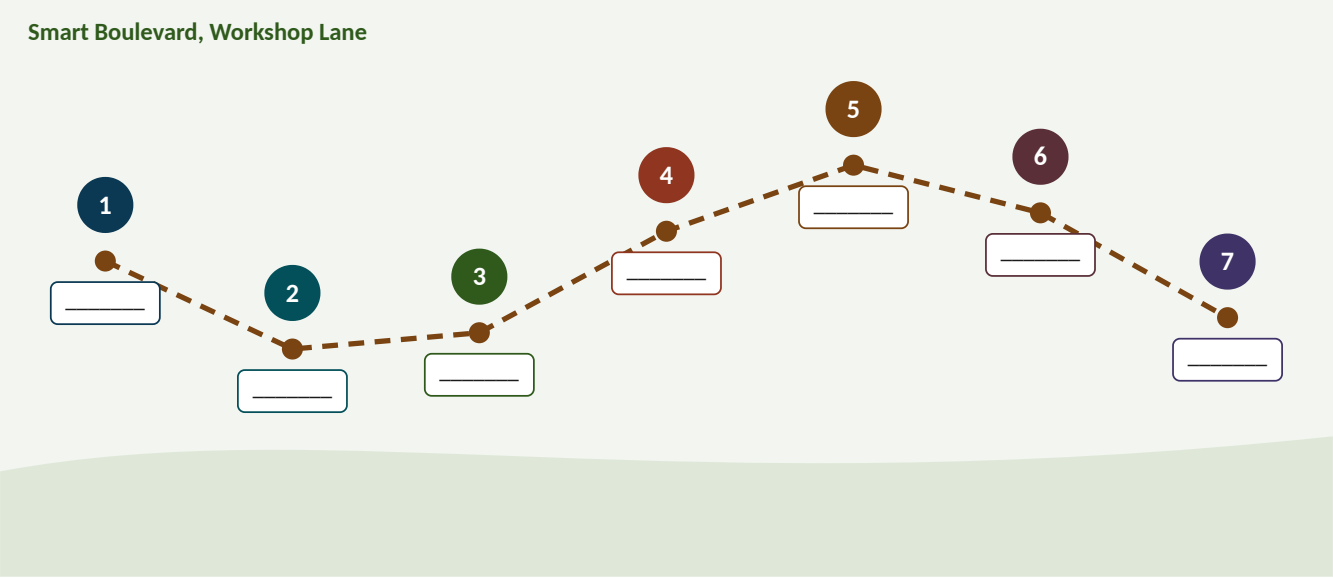
TEACH IT TO A 14-YEAR-OLD (10 min)

Explain to a 14-year-old how a traffic light can weigh who is waiting, learn from treats, tell its neighbour what is coming, and why a strict rule-checker must stand between a chatty AI and the lights.

Words you may not use: agent, hysteresis, reward, SPaT, shield

Fill the blank route

Write the concept held at each stop, and its key number.



One-page cheat sheet

1. Set-up and the Junction class

pip install -r requirements.txt; pytest checks without SUMO. TraCI steps SUMO each second. Junction.request(stage) → yellow 3 s → all-red 1 s → green. Min 8 s, max 60 s. 4 s lost per change.

Route: Toolbox shop

2. Lab 1: your first agent

Score = 1·queue + 0.7·arriving + 0.15·longest wait; switch if other > current + 2. Decides every 2 s. Delay: fixed 14.1, actuated 9.8, max-pressure 15.0, agent 9.6 s. Longest wait 36 / 26 / 301 / 20 s.

Route: First junction

3. Lab 2: many agents on a corridor

Corridor: 4 junctions, 300 m, 3 seeds. ETA = now + distance ÷ 12 m/s; +0.8 × size if due in 12 s. Delay: wave 26.5, actuated 17.5, MP 24.3, silent 18.4, talking 18.2 s. Stops 0.95 → 0.88.

Route: Four houses

4. Lab 3: a learning agent (Q-learning)

State = stage | EW bin | NS bin | green bin (bins of 3 cars). Keep or switch every 5 s. r = −(queue EW + NS). α 0.2, γ 0.9, ε 30% → 2%. 30 episodes × 30 min. 31 states; test 11.7 s vs 9.8 / 9.6 s.

Route: Dog school

5. Lab 4: eco-driving with SPaT and GLOSA

GLOSA range 300 m, min speed 5 m/s. Next green = remaining + 3 + 1 s. Fixed + GLOSA 0 → 100%: stops 1.12 → 0.50, CO_2 254.4 → 244.3 g. Agent (8 s commit): delay 24.0 → 20.0 s. No commit, same seed: 18.4 s.

Route: Swan parade

6. Lab 5: tools for priority

priority_requests(): buses and ambulances within 200 m. w_priority = 25. Bus every 3 min, ambulance every 5 min. Bus 33.4 / 29.1 / 7.8 s; ambulance 17.3 / 17.8 / 3.0 s; all 17.8 / 18.2 / 18.4 s.

Route: Hospital road

7. Lab 6: an LLM agent behind a safety shield

Mock model by default; anthropic backend for a real one. Decides every 5 s. Shield: unknown → keep; no switch < 8 s; force switch at 60 s or 90 s wait. Actuated 9.8, reliable 10.7, 20% bad + shield 11.6 s. 165 bad of 735 calls.

Route: Parrot cage

Numbers to remember

- Yellow 3 s, all-red 1 s, min green 8 s, max green 60 s
- Score = 1·queue + 0.7·arriving + 0.15·wait; switch if ahead by 2
- Lab 1 delay: 14.1 / 9.8 / 15.0 / 9.6 s; longest wait 301 s (MP) vs 20 s
- Lab 2: stops 0.95 → 0.88 with talking; actuated 17.5 s lowest
- Lab 3: α 0.2, γ 0.9; 31 states; 11.7 s vs 9.6 s rules
- Lab 4: v = d ÷ (remaining + 4 s); fixed stops 1.12 → 0.50
- Lab 5: w_priority 25; bus 33.4 → 7.8 s. Lab 6: 165 bad answers, 11.6 s

Answer key

Try it now

Try it 1 (Set-up and the Junction class): Fixed: 122 × 4 = **488 s**. Actuated: 254 × 4 = **1016 s**, about 17 minutes.

Try it 2 (Lab 1: your first agent): EW = 5 + 2.1 + 0.3 = 7.4; NS = 6 + 0 + 4.5 = 10.5. With hysteresis 2: 10.5 > 9.4, **switch**. With hysteresis 6: 10.5 < 13.4, **keep**.

Try it 3 (Lab 2: many agents on a corridor): 180 ÷ 12 = 15 s, so ETA = now + 15 s, just outside the 12 s window. About 3 s later it adds 0.8 × 4 = **3.2** points to EW.

Try it 4 (Lab 3: a learning agent (Q-learning)): Target = −4 + 0.9 × (−5) = −8.5; error = −8.5 − 0 = −8.5; step = 0.2 × (−8.5) = −1.7; new Q = **−1.7**.

Try it 5 (Lab 4: eco-driving with SPaT and GLOSA): At the limit: 250 ÷ 13.89 = 18.0 s. Next green in 20 + 4 = 24 s. 18 < 24, so advise 250 ÷ 24 = **10.4 m/s** (37.5 km/h).

Try it 6 (Lab 5: tools for priority): (17.28 − 3.0) ÷ 17.28 = 14.28 ÷ 17.28 = **0.83**, about 83%.

Try it 7 (Lab 6: an LLM agent behind a safety shield): At 6 s: 6 < 8 s minimum green, so **keep**. At 30 s: "green_NE" is not a known action, so **keep** (unless someone has waited over 90 s, which forces a switch).

Self-quiz

Q1: B. Yellow 3 s, then all-red 1 s, so every stage change loses 4 s. The values are the course's teaching settings. (Concept 1)

Q2: B. perceive() calls the tools, reason() scores keep vs switch and writes a reason, act() passes the choice through the shield. (Concept 2)

Q3: C. A "platoon" message carries the number of cars and their ETA, so the neighbour can hold or start the green. (Concept 3)

Q4: A. GLOSA advises a connected car on a speed that reaches the stop line on green. (Concept 5)

Q5: B. The default backend is a mock, so the lab is repeatable and needs no key. The anthropic backend option plugs in a real model. (Concept 7)

Q6: B. Same yellow, all-red and minimum green for everyone, so differences in results come from the decisions. (Concept 1)

Q7: B. 1 × 4 + 0.7 × 2 + 0.15 × 10 = 4 + 1.4 + 1.5 = 6.9. (Concept 2)

Q8: C. Max-pressure serves the larger pressure and ignores how long anyone has waited. The agent's wait weight and 90 s cap prevent this. (Concept 2)

Q9: B. Actuated 17.5 s, talking agents 18.2 s. Talking cut stops (0.95 → 0.88) but did not beat actuation on delay here. (Concept 3)

Q10: A. Target = −2 + 0.9 × (−5) = −6.5; error = −6.5 − (−4) = −2.5; step = 0.2 × (−2.5) = −0.5; new Q = −4.5. (Concept 4)

Q11: A. spat() returns the green stage but 'unknown' remaining time until the agent commits a change, here 8 s ahead. (Concept 5)

Q12: C. priority_requests() sees buses and ambulances within 200 m; w_priority = 25. (Concept 6)

Q13: C. 18.21 → 18.44 s, about +0.2 s, while bus delay fell from 29.1 to 7.8 s. (Concept 6)

Q14: C. Any action other than keep or switch is unknown and becomes keep (unless the 60 s or 90 s rules force a switch). (Concept 7)

Q15: B. Bar = 5.0 + 6 = 11.0, and 11.3 > 11.0, so it still switches, but only just. (Concept 2)

Q16: C. 2 × 6 × 6 × 4 = 288 possible; 31 visited is about 11%. Most combinations never happen at this demand. (Concept 4)

Q17: C. At the limit it arrives in 14.4 s; next green in 16 + 4 = 20 s; 200 ÷ 20 = 10.0 m/s. (Concept 5)

Q18: B. The score rule does not check minimum green itself; that rule lives in the shield, which blocked such switches. (Concept 7)

Draw it: the completed flow is Figure 1.

Your revision plan

When	What	Done
Today	Read one concept a sitting. Do every Try-it-now on paper.	☐
Day 1	Walk the route from memory. Redraw the master mind map.	☐
Day 3	Warm-up and core quiz without notes. Re-read what you missed.	☐
Day 7	Whole quiz. Do the 'build the smallest example' task.	☐
Day 14	Teach-it task out loud. Cheat sheet from memory, then compare.	☐
Before an exam or project	Cheat sheet plus route walk only.	☐

Going further

- Lopez, P.A. et al. (2018). Microscopic traffic simulation using SUMO. *IEEE ITSC 2018*. https://doi.org/10.1109/ITSC.2018.8569938
- Alegre, L.N. SUMO-RL: reinforcement learning environments for traffic signal control with SUMO (software repository). https://github.com/LucasAlegre/sumo-rl
- Varaiya, P. (2013). Max pressure control of a network of signalized intersections. *Transportation Research Part C*. https://doi.org/10.1016/j.trc.2013.08.014
- Lai, S., Xu, Z., Zhang, W., Liu, H. and Xiong, H. LLMLight: large language models as traffic signal control agents. arXiv:2312.16044. https://arxiv.org/abs/2312.16044
- Lebre, M.-A. et al. (2015). Real scenario and simulations on GLOSA traffic light system for reduced CO2 emissions, waiting time and travel time. arXiv:1506.01965. https://arxiv.org/abs/1506.01965
- This course: Go 2 page, memory route and playgrounds, smrahmanbd.github.io/agentic-atsc-lab; code and labs, https://github.com/smrahmanbd/agentic-atsc-lab

HONESTY NOTE

All results come from this repository's small synthetic SUMO scenarios (one junction or a four-junction corridor, Poisson demand, mostly seed 1; Lab 2 is a mean of 3 seeds) and illustrate the methods, not real-world performance. Several results are negative: actuated control still had the lowest delay on the Lab 2 corridor; Q-learning did not yet beat the rules; the 8 s commitment needed for GLOSA cost the agent delay (the 18.4 s no-commitment figure is the Lab 2 seed-1 run of the same corridor). The LLM in Lab 6 is a mock by default, so its numbers describe the shield, not any real language model. Goal weights and timings are the course's teaching values. Worked examples use invented observations chosen to be easy by hand.